DATA MINING AND ANALYTICS – COU 08104

QUIZ 3

STUDENT NAME: AHMADI ABASI AHMADI

REG: 230242419000

CLASS: BENG22COE

QN 1

Exercise

TID	Items
1	Bread, Milk, Chips, Mustard
2	Beer, Diaper, Bread, Eggs
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk, Chips
5	Coke, Bread, Diaper, Milk
6	Beer, Bread, Diaper, Milk, Mustard
7	Coke, Bread, Diaper, Milk

Generate frequent item sets given min_support=40%

SOLUTION:

CODE SNIPESTS

apriori.py > ...

```python
from itertools import combinations
import math

# Transaction data from the table
transactions = [
    ['Bread', 'Milk', 'Chips', 'Mustard'],
    ['Beer', 'Diaper', 'Bread', 'Eggs'],
    ['Beer', 'Coke', 'Diaper', 'Milk'],
    ['Beer', 'Bread', 'Diaper', 'Milk', 'Chips'],
    ['Coke', 'Bread', 'Diaper', 'Milk'],
    ['Beer', 'Bread', 'Diaper', 'Milk', 'Mustard'],
    ['Coke', 'Bread', 'Diaper', 'Milk']
]

# Parameters
min_support_percent = 40
num_transactions = len(transactions)
min_support_count = math.ceil((min_support_percent / 100) * num_transactions)

print(f"Number of transactions: {num_transactions}")
print(f"Minimum support count: {min_support_count} (40% of {num_transactions})\n")

# Get all unique items
all_items = set()
for transaction in transactions:
    all_items.update(transaction)
all_items = list(all_items)

print(f"All unique items: {sorted(all_items)}")

# Function to calculate support for itemsets
def calculate_support(itemset, transactions):
    count = 0
    for transaction in transactions:
```

apriori.py > ...

```python
print(f"All unique items: {sorted(all_items)}")

# Function to calculate support for itemsets
def calculate_support(itemset, transactions):
    count = 0
    for transaction in transactions:
        if set(itemset).issubset(set(transaction)):
            count += 1
    return count

# Step 1: Generate frequent 1-itemsets
frequent_itemsets = {}
candidate_1_itemsets = {}
for item in all_items:
    count = calculate_support([item], transactions)
    if count >= min_support_count:
        candidate_1_itemsets[frozenset([item])] = count

frequent_itemsets[1] = candidate_1_itemsets
print("\n=== Frequent 1-itemsets ===")
for itemset, support in sorted(frequent_itemsets[1].items()):
    print(f"{set(itemset)}: {support} ({support/num_transactions*100:.1f}%)")

# Function to generate candidate itemsets
def generate_candidates(prev_frequent_itemsets, k):
    candidates = []
    prev_itemsets = list(prev_frequent_itemsets.keys())

    for i in range(len(prev_itemsets)):
        for j in range(i+1, len(prev_itemsets)):
            itemset1 = list(prev_itemsets[i])
            itemset2 = list(prev_itemsets[j])

```

```
            # Check if first k-2 items are the same
            if sorted(itemset1[:-1]) == sorted(itemset2[:-1]):
                new_candidate = set(itemset1) | set(itemset2)
                candidates.append(frozenset(new_candidate))

    return candidates

# Function to prune candidates using Apriori property
def prune_candidates(candidates, prev_frequent_itemsets, k):
    pruned_candidates = []

    for candidate in candidates:
        candidate_list = list(candidate)
        all_subsets_frequent = True

        # Generate all k-1 subsets
        subsets = combinations(candidate_list, k-1)

        for subset in subsets:
            if frozenset(subset) not in prev_frequent_itemsets:
                all_subsets_frequent = False
                break

        if all_subsets_frequent:
            pruned_candidates.append(candidate)

    return pruned_candidates

# Continue finding frequent itemsets of larger sizes
k = 2
while True:
```

```python
    # Generate candidates
    candidates = generate_candidates(frequent_itemsets[k-1], k)

    # Prune candidates
    candidates = prune_candidates(candidates, frequent_itemsets[k-1], k)

    # Count support for candidates
    candidate_counts = {}
    for candidate in candidates:
        count = calculate_support(list(candidate), transactions)
        if count >= min_support_count:
            candidate_counts[candidate] = count

    if not candidate_counts:
        break

    frequent_itemsets[k] = candidate_counts
    k += 1

# Print all frequent itemsets
print("\n=== All Frequent Itemsets ===")
for k in frequent_itemsets:
    if k > 1:
        print(f"\n=== Frequent {k}-itemsets ===")
    for itemset, support in sorted(frequent_itemsets[k].items()):
        itemset_list = sorted(list(itemset))
        print(f"{itemset_list}: {support} ({support/num_transactions*100:.1f}%)")

# Find maximal frequent itemsets
print("\n=== Maximal Frequent Itemsets ===")
maximal_itemsets = []
for k in sorted(frequent_itemsets.keys(), reverse=True):
    for itemset in frequent_itemsets[k]:
```

apriori.py > ...

```python
            if is_maximal:
                maximal_itemsets.append((itemset, frequent_itemsets[k][itemset]))

for itemset, support in maximal_itemsets:
    print(f"{set(itemset)}: {support} ({support/num_transactions*100:.1f}%)")

# Find closed frequent itemsets
print("\n=== Closed Frequent Itemsets ===")
closed_itemsets = []
for k in frequent_itemsets:
    for itemset in frequent_itemsets[k]:
        is_closed = True
        # Check if any superset has the same support
        for other_k in frequent_itemsets:
            if other_k > k:
                for other_itemset in frequent_itemsets[other_k]:
                    if itemset.issubset(other_itemset) and frequent_itemsets[k][itemset] == frequent_itemsets[other_k][othe
                        is_closed = False
                        break
                if not is_closed:
                    break

        if is_closed:
            closed_itemsets.append((itemset, frequent_itemsets[k][itemset]))

for itemset, support in sorted(closed_itemsets, key=lambda x: len(x[0])):
    print(f"{set(itemset)}: {support} ({support/num_transactions*100:.1f}%)")

print("\n=== Summary ===")
total_frequent_itemsets = sum(len(frequent_itemsets[k]) for k in frequent_itemsets)
print(f"Total number of frequent itemsets: {total_frequent_itemsets}")
print(f"Number of maximal frequent itemsets: {len(maximal_itemsets)}")
print(f"Number of closed frequent itemsets: {len(closed_itemsets)}")
```

OUTPUT

```
PS C:\Users\ahmad\Desktop\BENG22COE\DATA MINING\ASSIGNMENT 02\Assignment02\Assignment02\ASS2> python apriori.py
Number of transactions: 7
Minimum support count: 3 (40% of 7)

All unique items: ['Beer', 'Bread', 'Chips', 'Coke', 'Diaper', 'Eggs', 'Milk', 'Mustard']

=== Frequent 1-itemsets ===
{'Milk'}: 6 (85.7%)
{'Bread'}: 6 (85.7%)
{'Beer'}: 4 (57.1%)
{'Coke'}: 3 (42.9%)
{'Diaper'}: 6 (85.7%)

=== All Frequent Itemsets ===
['Milk']: 6 (85.7%)
['Bread']: 6 (85.7%)
['Beer']: 4 (57.1%)
['Coke']: 3 (42.9%)
['Diaper']: 6 (85.7%)

=== Frequent 2-itemsets ===
['Bread', 'Milk']: 5 (71.4%)
['Beer', 'Milk']: 3 (42.9%)
['Coke', 'Milk']: 3 (42.9%)
['Diaper', 'Milk']: 5 (71.4%)
['Beer', 'Bread']: 3 (42.9%)
['Bread', 'Diaper']: 5 (71.4%)
['Beer', 'Diaper']: 4 (57.1%)
['Coke', 'Diaper']: 3 (42.9%)

=== Frequent 3-itemsets ===
['Bread', 'Diaper', 'Milk']: 4 (57.1%)
['Beer', 'Diaper', 'Milk']: 3 (42.9%)
['Coke', 'Diaper', 'Milk']: 3 (42.9%)
```

```
=== Maximal Frequent Itemsets ===
{'Milk', 'Bread', 'Diaper'}: 4 (57.1%)
{'Diaper', 'Milk', 'Beer'}: 3 (42.9%)
{'Milk', 'Coke', 'Diaper'}: 3 (42.9%)
{'Diaper', 'Bread', 'Beer'}: 3 (42.9%)

=== Closed Frequent Itemsets ===
{'Milk'}: 6 (85.7%)
{'Bread'}: 6 (85.7%)
{'Diaper'}: 6 (85.7%)
{'Bread'}: 6 (85.7%)
{'Diaper'}: 6 (85.7%)
{'Milk', 'Bread'}: 5 (71.4%)
{'Milk', 'Diaper'}: 5 (71.4%)
{'Bread', 'Diaper'}: 5 (71.4%)
{'Diaper', 'Beer'}: 4 (57.1%)
{'Milk', 'Bread', 'Diaper'}: 4 (57.1%)
{'Diaper', 'Milk', 'Beer'}: 3 (42.9%)
{'Milk', 'Coke', 'Diaper'}: 3 (42.9%)
{'Diaper', 'Bread', 'Beer'}: 3 (42.9%)

=== Summary ===
Total number of frequent itemsets: 17
Number of maximal frequent itemsets: 4
Number of closed frequent itemsets: 11
```

EXPLANATIONS

This code implements the Apriori algorithm to find frequent itemsets with minimum support of 40% (which corresponds to 3 transactions out of 7).

Output Analysis:

The algorithm will produce:

1. **Frequent 1-itemsets**: Individual items that appear in at least 3 transactions
2. **Frequent 2-itemsets**: Pairs of items that appear together in at least 3 transactions
3. **Frequent 3-itemsets**: Triples of items that appear together in at least 3 transactions
4. **Frequent 4-itemsets**: Quadruples of items that appear together in at least 3 transactions

Key features of the solution:

- Uses the Apriori principle: all subsets of a frequent itemset must also be frequent
- Implements candidate generation and runing
- Identifies maximal frequent itemsets (itemsets that are frequent but none of their supersets are frequent)
- Identifies closed frequent itemsets (itemsets where no superset has the same support)